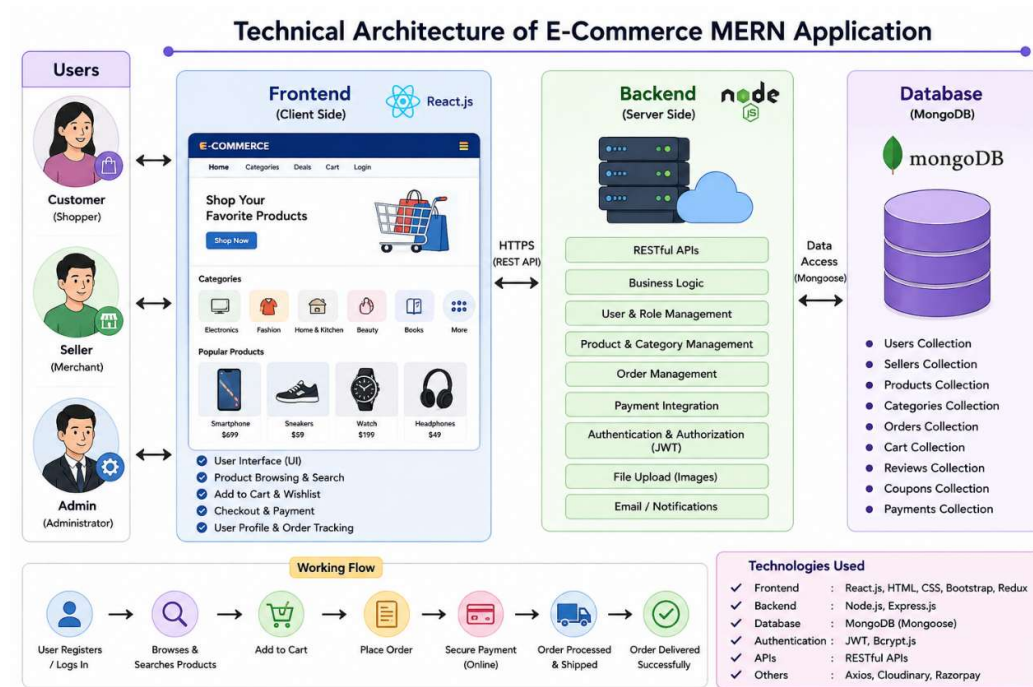


TECHNICAL ARCHITECTURE

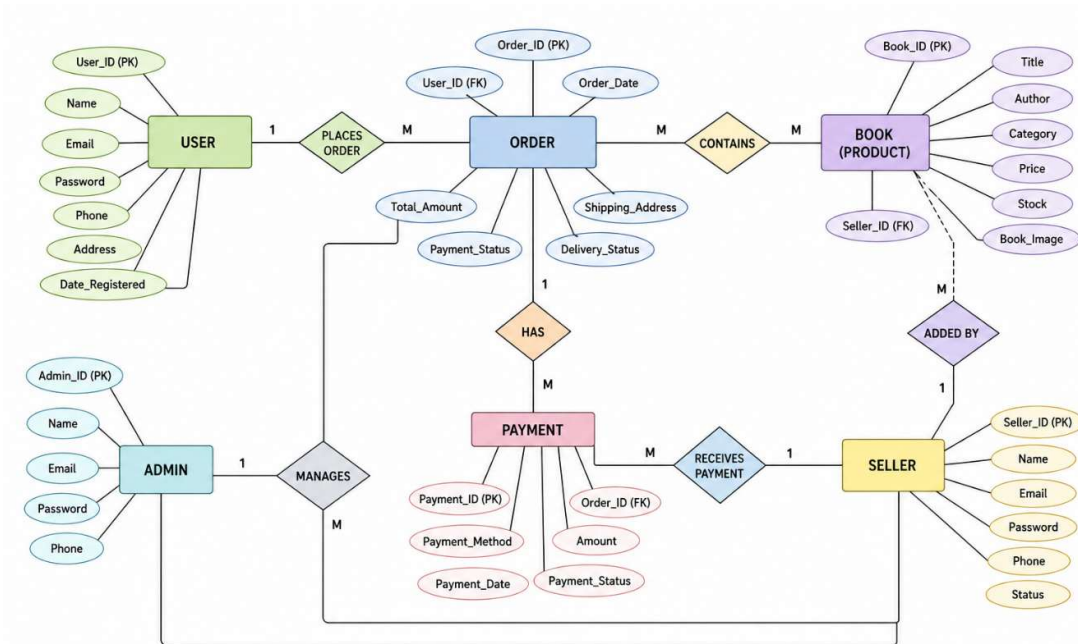


The **E-Commerce Application** follows a modern **client-server architecture** developed using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)**. The frontend acts as the client, providing a responsive and user-friendly interface that allows customers to browse products, search items, manage their shopping cart, place orders, and track their purchases. React.js is used to build reusable UI components, while HTML5, CSS3, Bootstrap, and JavaScript are used to create an attractive and responsive design. The frontend communicates with the backend through secure RESTful APIs, enabling smooth data exchange without reloading the entire webpage.

The backend of the application is developed using **Node.js** and **Express.js**, which handle all business logic and server-side operations. It processes user requests such as registration, login, product management, shopping cart operations, order placement, payment processing, and user profile management. The backend also performs authentication, authorization, validation, and error handling to ensure secure and reliable system functionality. REST APIs are implemented to enable seamless communication between the frontend and the database, providing efficient data processing and faster response times.

The application uses **MongoDB** as its NoSQL database to store user information, product details, categories, shopping cart data, orders, payment records, and customer reviews. **Mongoose** is used as the Object Data Modeling (ODM) library to define schemas and interact with the MongoDB database efficiently. Authentication is implemented using **JSON Web Tokens (JWT)**, ensuring secure user sessions and protecting sensitive information. The modular architecture separates the frontend, backend, and database into independent layers, making the application scalable, maintainable, secure, and easy to extend with future features such as online payment gateways, inventory management, product recommendations, and real-time order tracking.

ER-DIAGRAM



The **Entity Relationship (ER) Diagram** of the **E-Commerce Application** illustrates the logical structure of the database and defines how different entities interact with one another to support the complete online shopping process. The system consists of six major entities: **User**, **Seller**, **Admin**, **Book (Product)**, **Order**, and **Payment**. A **User** registers on the platform, logs in securely, browses available products, and places one or more orders. Each **Order** is associated with a specific user and contains information such as the order date, total amount, shipping address, payment status, and delivery status. Every order includes one or more **Books (Products)**, while each product is managed by a **Seller** who is responsible for adding, updating,

and maintaining product details such as title, author, category, price, stock availability, and product image. After an order is placed, the corresponding **Payment** entity stores transaction details including payment method, payment date, payment amount, and payment status, ensuring secure payment processing. The **Admin** oversees the entire platform by managing users, sellers, products, and overall system operations. The relationships shown in the ER diagram—such as **User Places Order**, **Order Contains Product**, **Order Has Payment**, **Seller Adds Product**, **Seller Receives Payment**, and **Admin Manages the System**—ensure proper data organization, maintain referential integrity through primary and foreign keys, minimize data redundancy, and provide efficient retrieval of information. This well-structured database design improves system scalability, simplifies maintenance, and supports reliable order management, inventory control, secure transactions, and overall application performance.

KEY FEATURES:

- 1. User Registration & Login:** The system allows customers, sellers, and administrators to securely register and log in using email and password.
- 2. User Profile Management:** Users can view and update their personal information such as name, email, phone number, and delivery address.
- 3. Product Browsing:** Customers can browse products displayed in different categories with attractive product cards. Each product contains images, descriptions, pricing, ratings, and stock information for better shopping experience.
- 4. Product Search & Filter:** The application provides a powerful search feature that helps users quickly find products by name or category. Filters such as price, category, brand, and availability improve product discovery.
- 5. Category Management:** Products are organized into categories like Electronics, Fashion, Home Appliances, and Books. This makes navigation simple and helps customers locate products efficiently.
- 6. Shopping Cart:** Customers can add products to the shopping cart before placing an order. They can also update quantities, remove unwanted items, and view the total cart amount instantly.

7. Wishlist: Users can save their favorite products in the wishlist for future purchases. This feature allows customers to revisit selected items without searching again.

8. Order Management: Customers can place orders after confirming their cart and shipping details. The system tracks order status such as Pending, Packed, Shipped, Delivered, or Cancelled.

9. Secure Payment: The application supports secure online payment gateways such as Razorpay or Stripe. Multiple payment methods like UPI, Credit Card, Debit Card, and Net Banking can be integrated.

10. Seller Dashboard: Sellers can add new products, update product details, manage inventory, and monitor customer orders. The dashboard provides complete control over their online store.

11. Product Management: Sellers and administrators can perform Create, Read, Update, and Delete (CRUD) operations on products.

12. Inventory Management: The system automatically updates product stock whenever an order is placed or cancelled.

13. Admin Dashboard: The administrator has complete control over the application by managing users, sellers, products, orders, and payments.

14. Order Tracking: Customers can track the progress of their orders from confirmation to delivery. Real-time order updates improve transparency and customer satisfaction.

15. Reviews & Ratings: Customers can provide ratings and reviews after purchasing products. These reviews help future buyers make informed purchasing decisions.

16. Discount & Coupons: The system allows administrators to create promotional offers, discount coupons, and seasonal sales.

17. Notification System: Email and in-app notifications are sent for order confirmation, payment success, shipping updates, and delivery completion. Sellers and administrators also receive important alerts.

18. Image Upload: Product images are uploaded and stored using cloud storage services like Cloudinary.

19. RESTful APIs: The backend is built using RESTful APIs that enable communication between the frontend and database. APIs handle authentication, product management, orders, payments, and user operations securely.

20. Database Management: MongoDB is used to store users, products, categories, orders, payments, reviews, and cart information. Mongoose provides schema validation and efficient database operations.

21. Authentication & Authorization: JWT-based authentication verifies user identity, while role-based authorization restricts access according to user roles such as Customer, Seller, or Admin. This improves application security.

22. Security Features: Passwords are encrypted using bcrypt before storing them in the database. Input validation, protected routes, and secure APIs safeguard the application against unauthorized access and common attacks.

23. Analytics & Reports: The administrator can monitor sales performance, product statistics, customer registrations, and revenue reports. These insights help in making better business decisions.

ROLES AND RESPONSIBILITIES

1. User Registration and Authentication

Users can create an account, log in securely, and access the application using authenticated credentials. Passwords are encrypted, and JWT authentication ensures secure user sessions.

2. Profile Management

Users can update their personal information, change passwords, and manage shipping addresses. This helps maintain accurate customer details for order processing.

3. Product Management

Administrators can add, update, and delete products from the catalog. Product information such as price, description, images, and stock is maintained efficiently.

4. Category Management

Administrators organize products into categories to improve product navigation and searching. Categories can be created, updated, or removed as required.

5. Product Browsing and Search

Customers can browse available products, search using keywords, and filter items based on categories, price, or popularity for a better shopping experience.

6. Shopping Cart Management

Users can add products to the shopping cart, modify quantities, or remove unwanted items before completing the purchase process.

7. Order Management

Customers can place orders, while administrators monitor, process, and update order statuses from pending to delivered, ensuring efficient order fulfillment.

8. Payment Management

The application supports secure payment processing and stores transaction details. Customers receive payment confirmation after successful transactions.

9. Inventory Management

Administrators monitor product availability and update stock levels regularly. This prevents customers from purchasing unavailable products.

10. Customer Review and Rating

Customers can provide ratings and reviews for purchased products. These reviews help other buyers make informed purchasing decisions.

11. Dashboard Monitoring

Administrators access a dashboard to monitor users, products, orders, sales performance, and overall business activities in real time.

12. Security and Authorization

The system protects sensitive information using encrypted passwords, JWT authentication, and role-based authorization to prevent unauthorized access.

13. Notification Management

Customers receive notifications related to order confirmation, payment status, shipping updates, and promotional offers, improving user engagement.

14. Report Generation

Administrators generate reports on sales, orders, customer activities, and inventory, helping analyze business performance and support decision-making.

15. System Maintenance and Administration

Administrators maintain the overall application by managing database backups, monitoring server performance, updating system configurations, and ensuring continuous platform availability.

USER FLOW

Customer Flow

Role: Browse products, purchase items, track orders, and manage account details.

Flow

Account Registration and Login

- Register using name, email, phone number, and password.
- Verify account (if email verification is enabled).
- Log in securely using registered credentials.
- Access the customer dashboard after successful authentication.

Product Browsing and Search

- Explore products by categories such as Electronics, Fashion, Home Appliances, Grocery, Accessories, etc.
- Search products by name, brand, or keyword.
- Apply filters based on category, price, ratings, and availability.
- View detailed product information, specifications, images, and customer reviews.

Shopping Cart and Checkout

- Add selected products to the shopping cart.
- Update product quantities or remove unwanted items.
- Review the cart summary before checkout.
- Enter shipping address and select the preferred payment method.
- Confirm and place the order securely.

Order Monitoring

- View all placed orders from the customer dashboard.
- Track order status such as **Pending, Confirmed, Processing, Shipped, Out for Delivery, and Delivered.**

- View order details, invoices, and payment status.

Payment and Notifications

- Complete payments using available payment methods.
- Receive order confirmation after successful payment.
- Get notifications regarding order processing, shipping updates, and delivery confirmation.

Profile Management

- Update personal information such as name, email, phone number, and address.
- Change account password securely.
- View previous orders and purchase history.
- Manage saved delivery addresses.

Seller Flow

Role: Manage products, process customer orders, and maintain inventory.

Flow

Registration and Login

- Register as a seller by providing business details and account information.
- Wait for administrator approval before accessing seller services.
- Log in securely using authorized credentials.

Product Management

- Add new products with product name, category, description, price, stock quantity, and product images.
- Edit existing product details whenever required.
- Remove discontinued or unavailable products from the catalog.

Order Management

- Access customer orders through the seller dashboard.

- View complete order details including customer information and purchased products.
- Accept and process incoming orders.

Inventory Management

- Update available stock after each successful order.
- Monitor inventory levels regularly.
- Mark products as out of stock when inventory is exhausted.

Order Processing

- Update order status from **Pending** → **Confirmed** → **Processing** → **Shipped** → **Delivered**.
- Generate shipping details and dispatch products.
- Handle order cancellations and return requests.

Customer Interaction

- Respond to customer queries regarding products and orders.
- Address customer complaints and provide necessary support.
- Ensure timely communication for better customer satisfaction.

ADMINISTRATOR

Role: Manage the entire E-Commerce platform, oversee users and sellers, monitor products and orders, and ensure the smooth operation of the system.

System Administration

- Monitor the overall activities of the E-Commerce platform.
- Ensure smooth functioning of the application and server.
- Supervise system performance and resolve operational issues.
- Maintain platform availability and reliability.

Seller Management

- Review seller registration requests.

- Approve or reject seller accounts based on verification.
- Manage seller permissions and access rights.
- Monitor seller performance and product listings.

User Management

- Monitor registered customer accounts.
- View customer activities and purchase history.
- Activate, deactivate, or restrict user accounts when necessary.
- Resolve user account-related issues and support requests.

Product Management

- Monitor all products listed by sellers.
- Approve, update, or remove product listings.
- Manage product categories and featured products.
- Ensure product information is accurate and complies with platform policies.

Order Oversight

- Monitor all customer orders across the platform.
- Track order progress from **Pending** → **Confirmed** → **Processing** → **Shipped** → **Delivered**.
- Handle order cancellations, returns, and refund requests.
- Resolve delivery-related issues and escalations.

Payment Monitoring

- Verify successful payment transactions.
- Monitor failed or pending payments.
- Maintain payment records for auditing and reporting.
- Handle payment-related disputes when required.

Inventory Monitoring

- Monitor product stock levels.
- Ensure sellers maintain accurate inventory.
- Prevent the sale of out-of-stock products.
- Generate low-stock alerts when necessary.

Reports and Analytics

- View sales reports and revenue statistics.
- Monitor daily, monthly, and yearly business performance.
- Analyze customer purchasing trends and seller performance.
- Generate reports for business decision-making.

Platform Maintenance

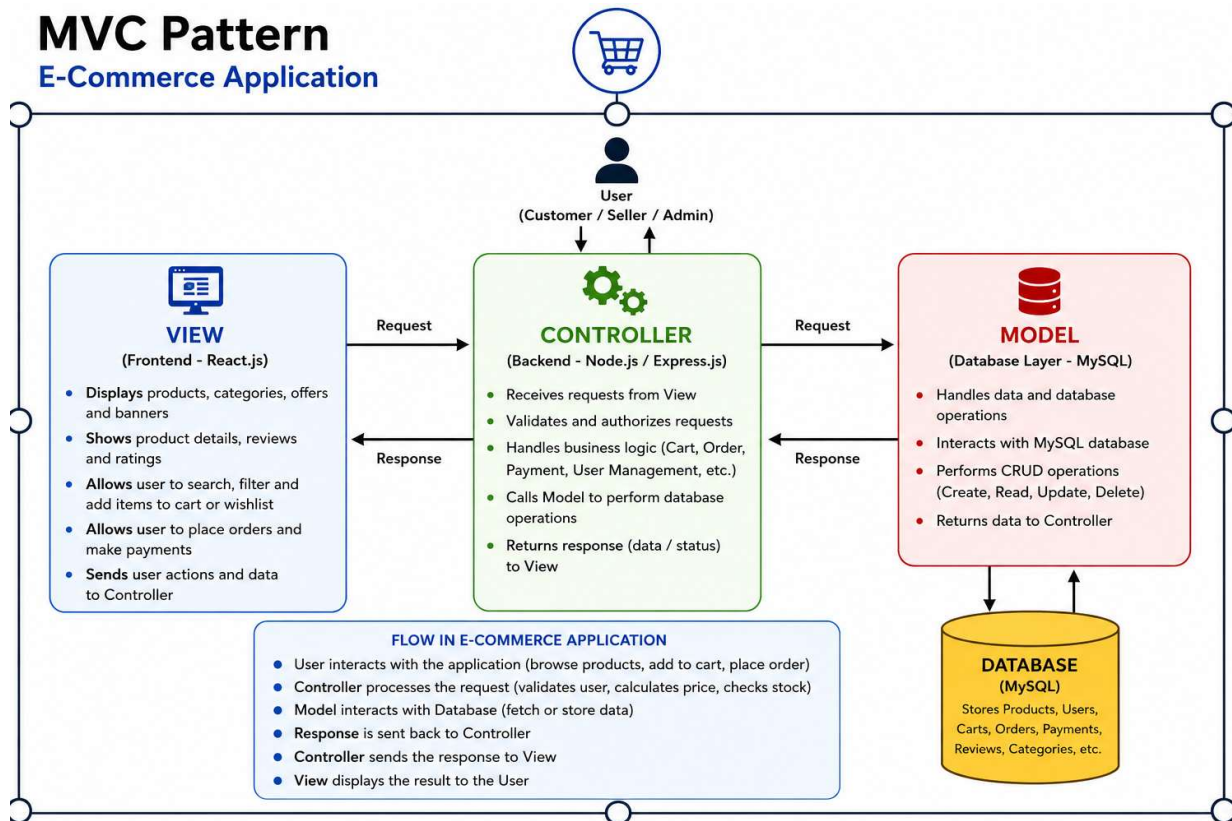
- Maintain application security and data protection.
- Perform database backup and recovery.
- Apply software updates and security patches.
- Improve system performance and fix technical issues.

Customer & Seller Support

- Handle customer complaints and support requests.
- Resolve seller-related issues and disputes.
- Ensure smooth communication between customers and sellers.
- Maintain overall platform quality and user satisfaction.

MVC Pattern

MVC Pattern E-Commerce Application



MVC Architecture

The **E-Commerce Application** is developed using the **Model-View-Controller (MVC)** architecture, which separates the application into three independent components: **Model**, **View**, and **Controller**. This architectural pattern improves code organization, simplifies maintenance, enhances scalability, and allows developers to manage the application efficiently. Each layer has a specific responsibility and communicates with the others through well-defined interfaces, resulting in a clean and structured application.

Model Layer (E-Commerce Application)

The **Model** layer is responsible for managing all application data and database operations. It defines the structure of entities such as **Users**, **Products**, **Categories**, **Shopping Cart**, **Orders**, **Payments**, **Reviews**, **Sellers**, and **Administrators**. The model performs all Create, Read,

Update, and Delete (CRUD) operations on the MongoDB database while maintaining data consistency and integrity.

The Model layer stores customer information, product details, inventory records, shopping cart data, order history, payment transactions, product reviews, and seller information. It also validates database operations and ensures secure data handling throughout the application.

Controller Layer (Business Logic)

The **Controller** layer acts as the bridge between the **View** and the **Model**. It receives requests from customers, sellers, and administrators, processes business logic, validates user input, and interacts with the database through the Model layer.

The Controller manages functionalities such as user registration and login, product search, category filtering, shopping cart operations, order placement, payment processing, inventory management, seller approvals, product management, review handling, and order tracking. After processing each request, the controller returns the appropriate response to the frontend for display.

View Layer (User Interface)

The **View** layer represents the user interface of the **E-Commerce Application**. It is developed using **React.js**, providing an interactive and responsive shopping experience for customers, sellers, and administrators.

The View layer includes pages such as **Home Page, User Registration, Login, Product Listing, Product Details, Shopping Cart, Checkout, Order Tracking, User Profile, Seller Dashboard, Product Management, Admin Dashboard, Category Management, Order Management, and Payment Confirmation**. It collects user input, sends requests to the Controller, and displays the processed information in a user-friendly format.

Advantages of MVC Architecture in the E-Commerce Application

1. Modular Design

Each component performs a specific responsibility, making the application well-organized, easy to understand, and simple to manage.

2. Easy Maintenance

Changes made in the Model, View, or Controller have minimal impact on the other layers, reducing development complexity and maintenance effort.

3. Scalability

New modules such as Wishlist, Coupons, Product Recommendations, Inventory Analytics, or Delivery Tracking can be integrated without affecting the existing architecture.

4. Code Reusability

Models, Controllers, and React components can be reused across multiple modules, reducing duplicate code and improving development efficiency.

5. Better Security

Business logic remains in the Controller, while database operations are handled through the Model, ensuring secure data access and preventing unauthorized operations.

6. Improved Development Efficiency

Frontend and backend developers can work independently on different layers, accelerating the development process.

7. Simplified Testing

Each layer can be tested independently, making it easier to identify and fix bugs while improving software quality.

8. Better User Experience

The separation of concerns enables faster response times, smoother navigation, and a highly responsive user interface.

CLIENT SETUP (installing react app)

Client Setup (React.js)

Setting Up the React Frontend (E-Commerce Application)

The frontend of the E-Commerce Application is developed using React.js with Vite as the build tool. Vite provides a faster development environment with

instant server startup, hot module replacement (HMR), and optimized production builds. The following steps explain how to set up the React frontend.

Description

Step 1: Open the Client Folder

- Open the client folder of the E-Commerce Application in Visual Studio Code.
- Launch the integrated terminal (Terminal → New Terminal).

Step 2: Create a React Project Using Vite

Execute the following command to create a React project:

```
npm create vite@latest . -- --template react
```

Step 3: Select Project Options

When prompted, choose the following options:

- Framework: React
- Variant: JavaScript

Step 4: Navigate to the Client Directory

Move to the client project folder:

```
cd client
```

Step 5: Install Required Dependencies

Install all required packages using:

```
npm install
```

This command installs all dependencies listed in the package.json file, including React, Vite, and other required frontend libraries.

Step 6: Start the React Development Server

Run the following command:

```
npm run dev
```

Step 7: Launch the Application

After the development server starts successfully, open the URL displayed in the terminal (typically <http://localhost:5173>) in your web browser.

Backend Server Configuration(Server Setup)

9.1 Setting Up the Node.js + Express Backend

The backend of the **E-Commerce Application** is developed using **Node.js** and **Express.js**. It manages business logic, authentication, REST APIs, database communication, product management, order processing, payment handling, and user management.

Step 1: Open the Server Folder

- Open the **server** folder of the **E-Commerce Application** in **Visual Studio Code**.
 - Launch the integrated terminal.
-

Step 2: Initialize a Node.js Project

Execute the following command:

```
npm init -y
```

This command creates a **package.json** file containing the project configuration, scripts, dependencies, and metadata required for the backend application.

Step 3: Create the Main Server File

Create the main backend entry file:

```
server.js
```

This file initializes the Express application, configures middleware, connects to MongoDB, and starts the backend server.

Step 4: Create Backend Folder Structure

Organize the backend by creating the following directories:

controllers/

models/

routes/

middleware/

config/

uploads/

Purpose of Each Folder

models/

Stores MongoDB schemas and data models for:

- Users
 - Sellers
 - Products
 - Categories
 - Shopping Cart
 - Orders
 - Payments
 - Reviews
-

controllers/

Implements the business logic for:

- User Registration and Login
 - Seller Authentication
 - Product Management
 - Category Management
 - Shopping Cart Operations
 - Order Processing
 - Payment Handling
 - Inventory Management
 - Review Management
 - Admin Dashboard Operations
-

routes/

Defines REST API endpoints for:

- Authentication
- Users
- Sellers
- Products
- Categories
- Shopping Cart
- Orders
- Payments
- Reviews
- Admin Management

Each route is connected to its corresponding controller.

middleware/

Contains middleware functions such as:

- JWT Authentication
- Authorization
- Error Handling
- Request Validation
- File Upload Middleware

config/

Stores configuration files including:

- MongoDB Connection
- Environment Variables
- JWT Configuration
- Application Settings

uploads/

Stores uploaded files such as:

- Product Images
- Seller Documents
- User Profile Images

Backend Development Ready

After creating the required files and folders, the backend structure is fully prepared for:

- Developing RESTful APIs
- Integrating MongoDB Database
- Implementing JWT Authentication
- Managing Products and Categories
- Shopping Cart Functionality
- Order Management
- Payment Processing
- Inventory Control
- Seller and Customer Management
- Review and Rating System
- Admin Dashboard Operations

This setup establishes a scalable and maintainable backend architecture for the **E-Commerce Application**, enabling secure communication between the React frontend and the MongoDB database through Express.js REST APIs.

DEVELOPMENT AND EXPLANATION

1. Project Setup

The development of the E-Commerce Application begins by creating the backend project and initializing it using the `npm init` command. Required packages such as Express.js, Mongoose, CORS, JWT, bcrypt, dotenv, and Nodemon are installed to support backend development and secure API communication.

2. Express Server Creation

An Express.js server is configured to handle client requests and provide RESTful API services. Middleware such as `express.json()`, CORS, and dotenv is integrated to process request data, manage environment variables, and enable seamless communication between the React frontend and the backend.

3. Route Development

Separate route files are created for modules such as Authentication, Users, Products, Categories, Cart, Orders, Payments, Sellers, Reviews, and Admin. These routes define the application's API endpoints and forward requests to their respective controllers.

4. Controller Implementation

Controllers are implemented to handle the application's business logic. They process user requests, perform validation, interact with MongoDB models, and return appropriate responses for operations such as authentication, product management, shopping cart, and order processing.

5. Database Model Creation

Mongoose schemas and models are created for entities including Users, Sellers, Products, Categories, Cart, Orders, Payments, Reviews, and Admins. These models define the database structure and enable efficient interaction with MongoDB.

6. Product Management

The product module enables sellers and administrators to add, update, delete, and manage products. It also maintains product categories, pricing, descriptions, images, and inventory information.

7. Order Management

The order module allows customers to place orders, view order details, and track delivery status. Sellers and administrators can process orders, update order statuses, and ensure timely delivery of products.

8. User Authentication

Authentication is implemented using JSON Web Tokens (JWT) and bcrypt password encryption. Customers, sellers, and administrators can securely register and log in, while protected routes are accessible only to authorized users.

9. Admin Operations

The admin module provides complete control over the platform by managing users, sellers, products, categories, orders, payments, and reports. Administrators can also monitor platform activities and maintain overall system security.

10. Shopping Cart Functionality

The shopping cart module enables customers to add products, update quantities, remove items, and review their cart before checkout. Cart information is securely stored and linked to the user's account for a seamless shopping experience.

11. Payment Processing

The payment module manages secure online transactions by supporting multiple payment methods. It records payment details, verifies transaction status, and generates payment confirmations for successful orders.

12. Review and Rating System

Customers can submit product reviews and ratings after purchasing items. This feature helps other users make informed buying decisions while improving product credibility and customer engagement.

13. Inventory Management

The inventory module continuously monitors product stock levels and automatically updates inventory after successful purchases. It prevents orders for unavailable products and ensures accurate stock management.

14. Seller Management

The seller module allows administrators to approve or reject seller registrations. Approved sellers can manage their products, process customer orders, and maintain inventory through the seller dashboard.

15. Frontend Integration

The React.js frontend communicates with the Node.js backend using REST APIs. Axios or Fetch API is used to exchange data, providing a responsive and real-time user experience across all modules.

CONFIGURE MONGODB

Install Mongoose

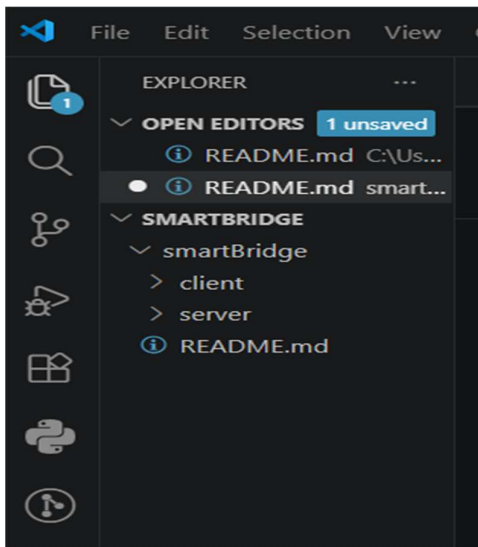
Mongoose is used to connect the Node.js application with the MongoDB database and perform database operations.

Run the following command in the terminal:

```
❑ npm install mongoose
```

After installation, Mongoose can be used to create database schemas, models, and connect the application to MongoDB.

CREATING FOLDER STRUCTURE

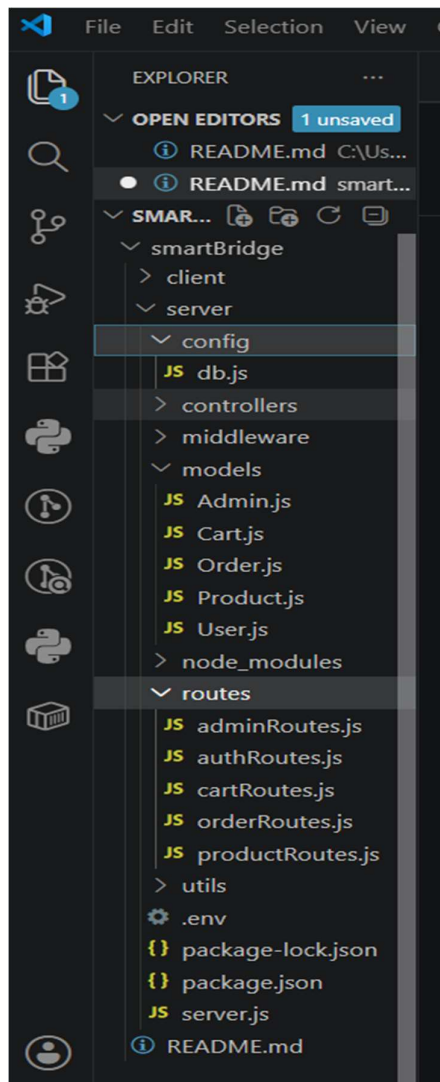


- The **E-Commerce Application** follows a well-organized directory structure that separates the **client (frontend)** and **server (backend)** components. This modular architecture improves code organization, simplifies maintenance, and allows the application to scale efficiently as new features are added.
- The **server** folder contains the complete backend implementation of the application. It includes directories such as **models** (for MongoDB schemas like Users, Products, Categories, Orders, Cart, Payments, Reviews, Sellers, and Admins), **controllers** (for implementing business logic such as authentication, product management, order processing, inventory management, and payment handling), and **routes** (for defining REST API endpoints). It also contains **middleware**, **configuration files**, and the main **server.js** file, which initializes the Express server and establishes the MongoDB database connection.

- The **client** folder contains the frontend of the application developed using **React.js**. It includes folders such as **components** (reusable UI components), **pages** (Home, Login, Registration, Product Listing, Product Details, Shopping Cart, Checkout, User Dashboard, Seller Dashboard, and Admin Dashboard), **assets** (images, icons, and static resources), and **styles** (CSS files). The frontend communicates with the backend through REST APIs to retrieve and display real-time data.

The separation of the frontend and backend enables independent development, easier debugging, better collaboration among developers, and improved scalability. This structured approach ensures that the **E-Commerce Application** remains maintainable, flexible, and ready for future enhancements.

BACKEND FOLDER STRUCTURE



- The **server** folder contains the complete server-side implementation of the **E-Commerce Application** and is responsible for handling client requests, executing business logic, and communicating with the MongoDB database. The **config** directory includes the **db.js** file, which establishes a secure connection with the MongoDB database. The **middleware** folder contains files such as **auth.js**, which implements **JWT (JSON Web Token)** authentication and authorization to provide secure access to protected resources and APIs.
- The **models** directory defines Mongoose schemas for entities such as **User, Seller, Product, Category, Cart, Order, Payment, Review, and Admin**, providing a structured representation of the application's data. The **routes** folder manages all RESTful API endpoints, including **user authentication, product management, category management, shopping cart operations, order processing, payment handling, seller management, review management, and administrator functionalities**, ensuring organized and efficient request processing.

The **server.js** file acts as the main entry point of the backend application by initializing the **Express.js** server, configuring middleware, connecting to MongoDB, and registering all API routes. Additionally, the **package.json** file manages project dependencies, scripts, and package information, while the **.env.example** file provides a template for environment variables such as database connection strings, JWT secrets, and server configuration, improving application security, configuration management, and deployment flexibility.

CREATE DATABASE CONNECTION

```
Book_Store > server > config > JS connect.js > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    try {
5      const conn = await mongoose.connect(process.env.MONGO_URI);
6      console.log(`MongoDB Connected: ${conn.connection.host}`);
7    } catch (error) {
8      console.error(`Error connecting to MongoDB: ${error.message}`);
9      process.exit(1);
10   }
11 };
12
13 module.exports = connectDB;
14
```

- The application connects to **MongoDB** using **Mongoose**, enabling efficient interaction with data such as Users, Books, and Orders.
- The database connection is configured using a **MongoDB URI stored in a .env file**, ensuring secure and reliable connectivity when the server starts.

SCHEMA AND MODELS

```
smartBridge > server > models > JS Admin.js > [Q] adminSchema
You, 5 days ago | 1 author (You)
1  const mongoose = require('mongoose');
2
3  const adminSchema = new mongoose.Schema(
4    {
5      banner: [
6        {
7          title: { type: String, required: true },
8          subtitle: { type: String, default: '' },
9          image: { type: String, required: true },
10         link: { type: String, default: '' },
11         isActive: { type: Boolean, default: true },
12       },
13     ],
14     categories: [
15       {
16         name: { type: String, required: true },
17         image: { type: String, default: '' },
18         description: { type: String, default: '' },
19         isActive: { type: Boolean, default: true },
20       },
21     ],
22   },
23   {
24     timestamps: true,
25   }
26 );
27
28 module.exports = mongoose.model('Admin', adminSchema);
29
```

```

smartBridge > server > models > JS Admin.js > adminSchema
You, 5 days ago | 1 author (You)
1  const mongoose = require('mongoose');
2
3  const adminSchema = new mongoose.Schema(
4  {
5      banner: [
6          {
7              title: { type: String, required: true },
8              subtitle: { type: String, default: '' },
9              image: { type: String, required: true },
10             link: { type: String, default: '' },
11             isActive: { type: Boolean, default: true },
12         },
13     ],
14     categories: [
15         {
16             name: { type: String, required: true },
17             image: { type: String, default: '' },
18             description: { type: String, default: '' },
19             isActive: { type: Boolean, default: true },
20         },
21     ],
22 },
23 {
24     timestamps: true,
25 }
26 );
27
28 module.exports = mongoose.model('Admin', adminSchema);
29

```

```

smartBridge > server > models > JS Users.js > userSchema > wishlist
You, 5 days ago | 1 author (You)
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs');
3
4  const userSchema = new mongoose.Schema(
5  {
6      name: { type: String, required: [true, 'Name is required'], trim: true, maxlength: [50, 'Name cannot exceed 50 characters'], },
7      email: { type: String, required: [true, 'Email is required'], unique: true, lowercase: true, trim: true, match: [/^\s*@(?!\.)([a-zA-Z\d-]+\.){1,63}$/i, 'Please enter a valid email'], },
8      password: { type: String, required: [true, 'Password is required'], minlength: [6, 'Password must be at least 6 characters'], select: false, },
9      mobile: { type: String, trim: true, default: '' },
10     role: { type: String, enum: ['user', 'admin'], default: 'user' },
11     addresses: [
12         {
13             fullName: { type: String, required: true }, phone: { type: String, required: true }, street: { type: String, required: true },
14             city: { type: String, required: true }, state: { type: String, required: true }, zipCode: { type: String, required: true },
15             country: { type: String, default: 'India' }, isDefault: { type: Boolean, default: false },
16         },
17     ],
18     wishlist: [
19         { type: mongoose.Schema.Types.ObjectId, ref: 'Product' },
20     ],
21 },
22 {
23     timestamps: true,
24 }
25 );
26
27 userSchema.pre('save', async function (next) {
28     if (!this.isModified('password')) return next();
29     const salt = await bcrypt.genSalt(12);
30     this.password = await bcrypt.hash(this.password, salt);
31     next();
32 });
33
34 userSchema.methods.comparePassword = async function (candidatePassword) {
35     return await bcrypt.compare(candidatePassword, this.password);
36 };
37
38 module.exports = mongoose.model('User', userSchema);
39

```

```

smartBridge > server > models > JS Orders > ordersSchema > shippingCost
You, 5 days ago | 1 author (You)
1 const mongoose = require('mongoose');
2
3 const orderSchema = new mongoose.Schema(
4 {
5   userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
6   products: [
7     {
8       productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true }, title: { type: String, required: true },
9       price: { type: Number, required: true }, quantity: { type: Number, required: true, min: 1 }, image: { type: String, default: '' },
10      shippingAddress: { fullName: { type: String, required: true }, phone: { type: String, required: true },
11      street: { type: String, required: true }, city: { type: String, required: true }, state: { type: String, required: true },
12      zipCode: { type: String, required: true }, country: { type: String, default: 'India' },
13      paymentMethod: { type: String, enum: ['COD', 'UPI', 'Card'], required: true },
14      total: { type: Number, required: true, min: 0 },
15      subtotal: { type: Number, required: true, min: 0 },
16      shippingCost: { type: Number, default: 0 },
17      status: {
18        type: String,
19        enum: ['Processing', 'Confirmed', 'Shipped', 'Delivered', 'Cancelled'],
20        default: 'Processing',
21      },
22      orderedDate: {
23        type: Date,
24        default: Date.now,
25      },
26    },
27    { timestamps: true,
28  }
29 );
30
31 orderSchema.index({ userId: 1 });
32 orderSchema.index({ status: 1 });
33 orderSchema.index({ orderedDate: -1 });
34
35 module.exports = mongoose.model('Order', orderSchema);
36

```

```

smartBridge > server > models > JS Products > ...
You, 5 days ago | 1 author (You)
1 const mongoose = require('mongoose');
2
3 const reviewSchema = new mongoose.Schema(
4 {
5   user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
6   name: { type: String, required: true },
7   rating: { type: Number, required: true, min: 1, max: 5 },
8   comment: { type: String, required: true },
9 },
10 { timestamps: true }
11 );
12
13 const productSchema = new mongoose.Schema(
14 {
15   title: { type: String, required: [true, 'Product title is required'], trim: true, maxlength: [200, 'Title cannot exceed 200 characters'], },
16   description: { type: String, required: [true, 'Product description is required'], maxlength: [2000, 'Description cannot exceed 2000 characters'], },
17   price: { type: Number, required: [true, 'Price is required'], min: [0, 'Price cannot be negative'], },
18   discount: { type: Number, default: 0, min: [0, 'Discount cannot be negative'], max: [100, 'Discount cannot exceed 100%'], },
19   stock: { type: Number, required: [true, 'Stock is required'], min: [0, 'Stock cannot be negative'], default: 0 },
20   category: { type: String, required: [true, 'Category is required'], trim: true },
21   brand: { type: String, trim: true, default: '' },
22   images: [{ type: String, }, ],
23   rating: { type: Number, default: 0, min: 0, max: 5 },
24   numReviews: { type: Number, default: 0 },
25   reviews: [reviewSchema],
26   isFeatured: { type: Boolean, default: false },
27   isTrending: { type: Boolean, default: false },
28 },
29 { timestamps: true,
30 }
31 );
32
33 productSchema.index({ title: 'text', description: 'text', category: 'text' });
34 productSchema.index({ category: 1 });
35 productSchema.index({ price: 1 });
36 productSchema.index({ rating: -1 });
37
38 module.exports = mongoose.model('Product', productSchema);
39

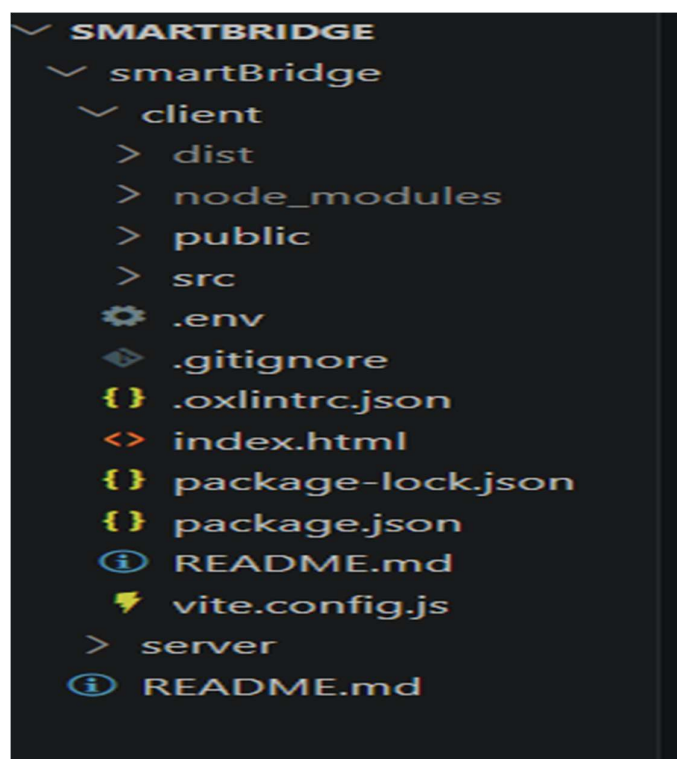
```

- **Imports Mongoose**, which provides schema-based modeling and simplifies MongoDB database operations in the **E-Commerce Application**..
- **Declares schemas** for different entities such as **User, Seller, Product, Category, Cart, Order, Payment, Review, and Admin**, defining the structure and relationships of application data.
- **Each schema contains essential fields** such as user information, product details (name, category, price, stock, description, images), order details, payment information, seller data, and customer reviews.
- **Mongoose models are created** using the `mongoose.model()` method, allowing the application to interact with MongoDB collections through structured objects.

- **These models are utilized in controllers** to perform operations such as user registration, product management, shopping cart handling, order placement, payment processing, and inventory updates.
- **Validation rules** are defined within schemas to ensure correct data formats, required fields, unique values, and overall data integrity before storing records in the database.
- **Error handling mechanisms** are implemented to capture and manage database operation failures, ensuring reliable application performance and meaningful error responses.
- **Models are exported** from their respective files so they can be reused across different modules such as controllers, routes, middleware, and other backend components.

The server imports these models during runtime through files like **server.js** and controller modules, enabling seamless communication between the Express.js backend and the MongoDB database.

FRONTEND FOLDER STRUCTURE



1. Setup React Application

The **client** folder contains the complete client-side implementation of the **E-Commerce Application**, developed using **React.js**. The application is organized inside the **src** directory

to ensure a clean, reusable, and scalable project structure. The **api** folder contains **axios.js**, which manages communication between the React frontend and the backend REST APIs. The **components** folder stores reusable UI components such as **Navbar.js**, **Footer.js**, **ProductCard.js**, and **ProtectedRoute.js** for secure page access. The **context** folder contains **AuthContext.js**, which manages user authentication and application state throughout the project.

The **pages** folder includes screen-level components such as **Home**, **Login**, **Register**, **Products**, **Product Details**, **Cart**, **Checkout**, **Orders**, **User Dashboard**, **Seller Dashboard**, and **Admin Dashboard**. The **App.js** file manages application routing, while **main.jsx** (or **index.js**, depending on the project setup) renders the React application in the browser. Global styling is maintained through **index.css**, and **package.json** stores all required frontend dependencies and scripts.

2. Design UI Components

The user interface is developed using **React.js** to provide a modern, responsive, and user-friendly shopping experience. Reusable components such as **Navbar**, **Footer**, **Product Cards**, **Category Cards**, **Search Bar**, and **ProtectedRoute** ensure consistency across the application. The **Navbar** enables smooth navigation between pages including **Home**, **Products**, **Categories**, **Cart**, **Orders**, **Profile**, **Seller Dashboard**, and **Admin Dashboard**, while **ProtectedRoute** restricts access to authenticated users only.

The application follows a component-based architecture where each page is responsible for a specific feature such as product browsing, shopping cart management, checkout, order tracking, seller product management, and administrator controls. This modular design improves code reusability, maintainability, responsiveness, and provides a consistent user experience across different devices.

3. Implement Frontend Logic

The frontend logic connects the React user interface with the backend services using **Axios** through the **axios.js** configuration file. It handles API requests for user registration, login, product retrieval, category browsing, shopping cart operations, checkout, order management, payment processing, seller operations, and administrator functionalities.

The **AuthContext.js** file manages authentication state across the application by storing user information and controlling login, logout, and authorization features. The **App.js** file handles

application routing by connecting public and protected pages, ensuring secure navigation and proper access control. This architecture enables efficient communication between the frontend and backend while providing a seamless, secure, and interactive shopping experience for customers, sellers, and administrators.

Project Execution

Steps for Project Execution:

Step 1: Download the Project

cd "C:\Users\admin\Downloads\Telegram Desktop\smartBridge\smartBridge"**Step 2: Install Frontend Dependencies**

```
cd client
```

```
npm install
```

```
npm run dev
```

Step 3: Install Backend Dependencies

```
cd ./server
```

```
npm install
```

```
npm start
```

Step 4: Create a .env file with your MongoDB URI:

```
PORT=5000
```

```
MONGO_URI=mongodb://localhost:27017/shopez
```

```
JWT_SECRET=shopez_jwt_secret_key_2024_production
```

```
JWT_EXPIRE=7d
```

```
NODE_ENV=development
```

```
CLIENT_URL=http://localhost:5173
```

DEMO SCREENSHOTS

S

ShopEZ

HomeProductsCategories

Search products...

Sign In

S

ShopEZ

Create Account

Join ShopEZ for the best shopping experience

Full Name

John Doe

Email

you@example.com

Mobile (Optional)

+91 9876543210

Password

Min. 6 characters

Create Account

- The Create Account page allows new users to securely register by entering their personal details, enabling access to all features of the ShopEZ e-commerce application.

S

ShopEZ

HomeProductsCategories

Search products...

Sign In

S

ShopEZ

Welcome back

Sign in to your account to continue

Email

you@example.com

Password

Enter your password

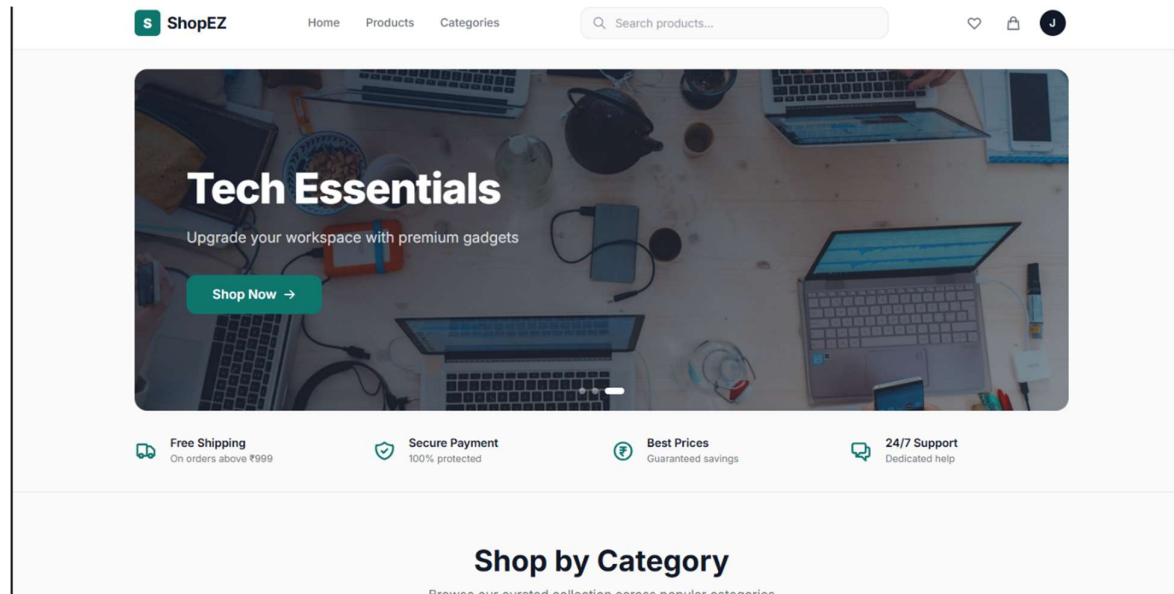
Sign In

Don't have an account? [Create one](#)

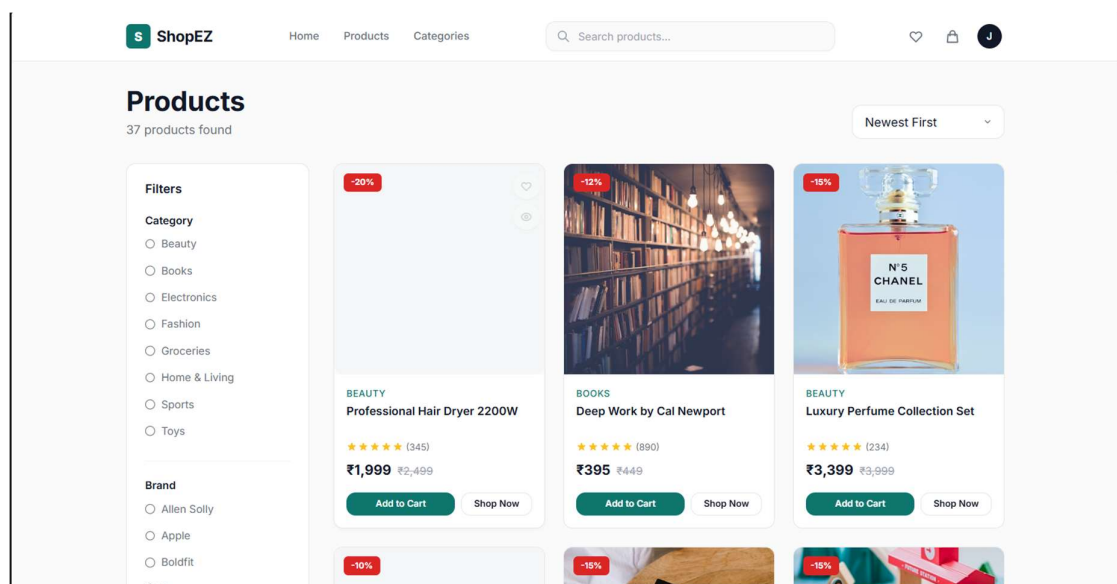
Demo Credentials
User: john@example.com / password123
Admin: admin@shopez.com / admin123

- **Login Page:**

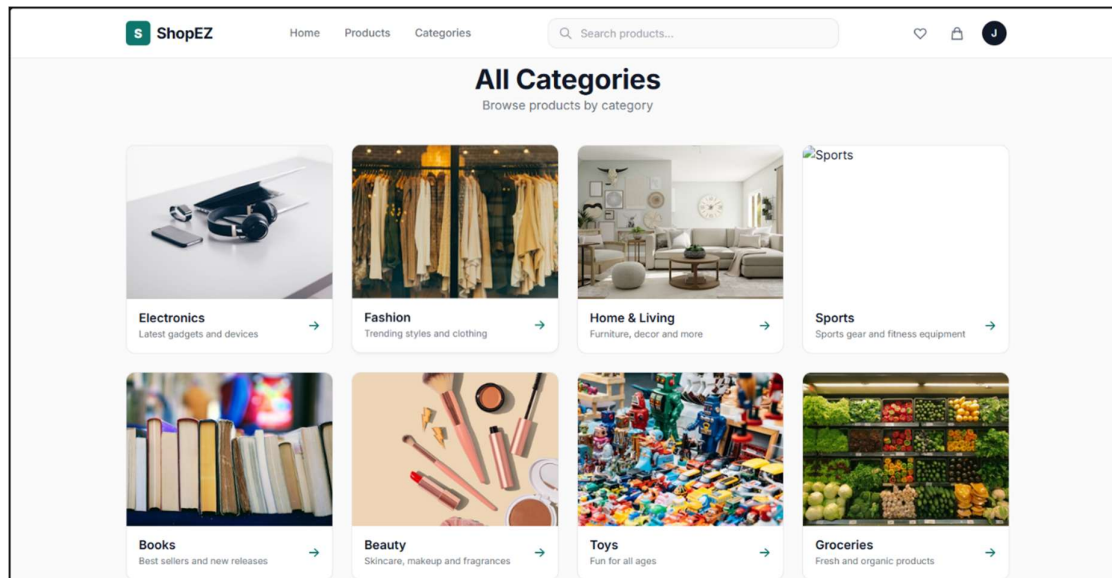
The **Login** page allows registered users to securely sign in using their email address and password to access their ShopEZ account and its features.



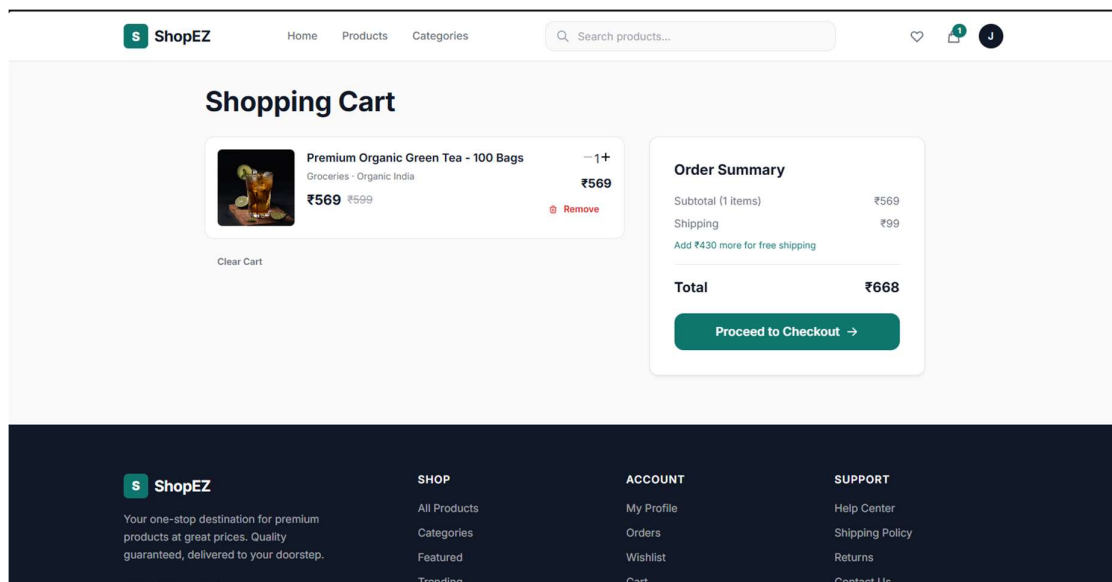
- The Home page provides users with featured products, categories, promotional banners, and quick access to all shopping services, ensuring a smooth and engaging shopping experience.



- The Product page displays all available products with filtering, sorting, pricing, ratings, and quick options to add items to the cart or purchase them.



The Categories page displays all product categories, allowing users to quickly browse and access products based on their interests.



The Add to Cart page allows users to review selected products, update quantities, and proceed to secure checkout for order placement.

DEMO LINKS

<https://drive.google.com/file/d/1jQV46CWxb3BsQPMO8xEPaLRuaSUU4P-G/view>